

I. LOAD BALANCING ALGORITHMS

A. EPLB Algorithm

The Energy-Proportional Load Balancing (EPLB) algorithm aims to distribute incoming inference requests across heterogeneous compute resources (CPU, NVIDIA L4, and NVIDIA H100) in a manner that minimizes total energy consumption while satisfying a defined latency constraint. This process is illustrated in the following pseudo code Algorithm 1. This pseudo code provides a structured, step-by-step view of how the EPLB strategy dynamically balances performance and power efficiency across diverse hardware types.

Algorithm 1: Energy-Proportional Load Balancing (EPLB)

Input: Request rate rps , latency limit l_{max}
Output: Request distribution among {CPU, L4, H100}
Initialize distribution $\leftarrow \{CPU : 0, L4 : 0, H100 : 0\}$;
remaining $\leftarrow rps$;
energy_ranking $\leftarrow []$;
foreach $inst \in \{CPU, L4, H100\}$ **do**
 $base_lat \leftarrow INSTANCE_SPECS[inst].base_inference_time_ms$;
 if $base_lat \leq l_{max}$ **then**
 $power \leftarrow INSTANCE_SPECS[inst].active_power_w$;
 $e_{req} \leftarrow power \times base_lat / 3,600,000$;
 Append $(inst, e_{req})$ to energy_ranking;
 end
end
Sort energy_ranking by ascending e_{req} ;
foreach $(inst, e_{req})$ in energy_ranking **do**
 if remaining ≤ 0 **then**
 break
 end
 $cap \leftarrow INSTANCE_POOL[inst].total_capacity_rps$;
 $alloc \leftarrow \min(remaining, cap)$;
 distribution[inst] $\leftarrow alloc$;
 remaining $\leftarrow remaining - alloc$;
end
if remaining > 0 **then**
 distribution[H100] $\leftarrow remaining$;
end
return distribution;

B. Round Robin Algorithm

The Round Robin Algorithm [1] is a simple and widely used technique in which each incoming request is assigned to the next available server in a circular sequence, ensuring that all servers share the load approximately equally over time. This algorithm calculates the total combined capacity of all instances. For each instance type, it computes its relative share (proportion) of the total capacity. It then assigns a proportional number of requests based on this share. Round Robin is defined below in this simulation process.

```
def round_robin_algorithm_realistic(request_count_per_sec):
    """Distribute proportionally to capacity"""
    total_capacity = sum(p['total_capacity_rps'] for p in INSTANCE_POOL.values())
    distribution = {}
    for inst_type, pool in INSTANCE_POOL.items():
        proportion = pool['total_capacity_rps'] / total_capacity
        distribution[inst_type] = request_count_per_sec * proportion
    return distribution
```

request_count_per_sec: The total number of incoming requests per second that need to be distributed.

INSTANCE_POOL: A global dictionary representing available server instances, where each entry includes the server's total processing capacity (total_capacity_rps).

C. Least Connections Algorithm

The Least Connections algorithm [2] is a technique that routes incoming requests to the server with the fewest active connections. This ensures that servers with a lighter load receive new requests.

```
def least_connections_algorithm_realistic(request_count_per_sec):
    """Round Robin with +/- 5% random variation"""
    rr_dist = round_robin_algorithm_realistic(request_count_per_sec)
    distribution = {}
    for inst_type, count in rr_dist.items():
        variation = np.random.uniform(-0.05, 0.05)
        distribution[inst_type] = max(0, count * (1 + variation))

    # Normalize
    total = sum(distribution.values())
    if total > 0:
        for inst_type in distribution:
            distribution[inst_type] = (distribution[inst_type] / total) * request_count_per_sec

    return distribution
```

The `least_connections_algorithm_realistic` function first calls the `round_robin_algorithm_realistic` function to obtain a base distribution of requests proportional to each server's capacity. A Small amount of random variation of $\pm 5\%$ per server was introduced to simulate real-world fluctuations in load, ensuring no server receives a negative number of requests. Since these variations can change the total request count, the function normalizes the values so that the sum of requests across all servers still equals the original `request_count_per_sec`.

D. Thermal-Aware Load Balancing Algorithm

A Thermal-aware Load Balancing Algorithm is a resource management strategy that sends the load based on current or predicted temperature. This algorithm predicts the temperature of the instance type as shown below.

```
MAX_TEMP_THRESHOLD = 80.0 # realistic datacenter limit

def predict_temperature(inst_type, utilization_pct):
    """Predict instance temperature based on utilization"""
    specs = INSTANCE_SPECS[inst_type]
    return (
        specs['base_temp_c'] +
        utilization_pct * specs['temp_increase_per_util']
    )
```

After estimating the utilization, assuming it handles the remaining load and filters out that instance type if it exceeds a maximum safe temperature threshold. The remaining instance types are sorted by lowest predicted temperature, and requests are sent to the coolest hardware first. If all options are exhausted and if the requests are still coming through, the algorithm sends the load to the H100 by default. The algorithm is defined as shown below.

```
def thermal_aware_algorithm_realistic(request_count_per_sec):
    """Lowest-Temperature-First Thermal-Aware Load Balancing"""
    distribution = {'CPU': 0, 'L4': 0, 'H100': 0}
    remaining = request_count_per_sec
    candidates = []
    for inst_type, pool in INSTANCE_POOL.items():
        capacity = pool['total_capacity_rps']
        utilization = min(100, (remaining / capacity) * 100)
        predicted_temp = predict_temperature(inst_type, utilization)
        if predicted_temp <= MAX_TEMP_THRESHOLD:
            candidates.append((predicted_temp, inst_type, capacity))
            candidates.sort(key=lambda x: x[0])
    for _, inst_type, capacity in candidates:
        if remaining <= 0:
            break
        alloc = min(remaining, capacity)
        distribution[inst_type] += alloc
        remaining -= alloc
    if remaining > 0:
        distribution['H100'] += remaining
    return distribution
```

E. Path Based Algorithm

This Path Based Algorithm performs load balancing by splitting incoming requests based on request paths as shown in the code below.

```
def split_requests_by_path(rps):
    """Split incoming requests per second by predefined path ratios"""
    return {
        path: rps * ratio
        for path, ratio in PATH_CLASSES.items()
    }
```

where path classes are defined as shown below to logically split by request type.

```
PATH_CLASSES = {
    'api': 0.6,          # 60% API traffic
    'admin': 0.25,       # 25% admin traffic
    'default': 0.15      # 15% other traffic
}
```

It then routes those requests to instance types based on the routing rules. Routing rules used in this study are

```
PATH_ROUTING_RULES = {
    'api': ['L4', 'H100'], # latency-sensitive
    'admin': ['CPU', 'L4'], # control / admin traffic
    'default': ['CPU']      # background / static
}
```

The algorithm then calculates the combined processing capacity of those eligible instance types, and distributes the path's requests among them in proportion to their individual capacities. As a result, higher-capacity resources receive a larger share of the traffic while lower-capacity resources receive less.

```
def path_based_algorithm_realistic(request_count_per_sec):
    """Path-Based Load Balancing with capacity-weighted routing"""
    distribution = {'CPU': 0, 'L4': 0, 'H100': 0}
    path_requests = split_requests_by_path(request_count_per_sec)
    for path, rps in path_requests.items():
        target_pools = PATH_ROUTING_RULES[path]
        total_capacity = sum(
            INSTANCE_POOL[p]['total_capacity_rps'] for p in target_pools
        )
        for inst_type in target_pools:
            pool_capacity = INSTANCE_POOL[inst_type]['total_capacity_rps']
            share = pool_capacity / total_capacity
            distribution[inst_type] += rps * share
    return distribution
```